

ĐỀ BÀI THỰC HÀNH

Phân lớp chữ số viết tay MNIST sử dụng Softmax Regression (NumPy)

Học máy 67PM1 & PM2

1 Mục tiêu

- Hiểu và cài đặt mô hình **Softmax Regression**.
- Huấn luyện mô hình trên tập MNIST.
- Tự viết code để đánh giá kết quả bằng các thước đo phổ biến trong phân lớp đa lớp.

2 Mô tả dữ liệu

- Input: vector ảnh có kích thước **(784,)** (flatten từ ảnh 28×28).
- Output: vector one-hot **(10,)** (các lớp số từ 0–9).
- Trọng số mô hình:
 - W : (784, 10)
 - b : (10,)

3 Yêu cầu

1. Hoàn thiện các hàm: **softmax**, **predict**, **cross_entropy**, **gradient update**.
2. Huấn luyện mô hình trên MNIST.
3. Sau khi train xong, viết các hàm **tự tính**:
 - Confusion matrix
 - Precision, Recall, F1-score (theo từng lớp và trung bình macro)
4. In ra kết quả đánh giá mô hình.

4 Khung Code

```
import numpy as np
from keras.datasets import mnist      # chi dung de load du lieu

# =====
# 1. Load du lieu
# =====
(x_train, y_train), (x_test, y_test) = mnist.load_data()

x_train = x_train.reshape(-1, 28*28) / 255.0      # (60000, 784)
x_test  = x_test.reshape(-1, 28*28) / 255.0       # (10000, 784)

num_classes = 10
y_train_oh = np.eye(num_classes)[y_train]         # (60000, 10)
y_test_oh  = np.eye(num_classes)[y_test]          # (10000, 10)

# =====
# 2. Khoi tao tham so
# =====
n_samples, n_features = x_train.shape             # (60000, 784)
W = np.zeros((n_features, num_classes))          # (784, 10)
b = np.zeros((num_classes,))                      # (10, )

# =====
# 3. Softmax
# =====
def softmax(z):
    """
    z: (N, 10)
    return: (N, 10)
    """
    # TODO: implement softmax
    pass

# =====
# 4. Du doan
# =====
def predict(X, W, b):
    """
    X: (N, 784)
    return: (N, 10) xac suat du doan
    """
    # TODO: tinh logits = XW + b, roi softmax
    pass

# =====
# 5. Loss function
# =====
def cross_entropy(y_true, y_pred):
    """
```

```

    y_true: (N, 10)
    y_pred: (N, 10)
    return: scalar loss
    """
    # TODO: implement cross-entropy
    pass

# =====
# 6. Training loop
# =====
lr = 0.1
epochs = 10
batch_size = 128

for epoch in range(epochs):
    for i in range(0, n_samples, batch_size):
        X_batch = x_train[i:i+batch_size]      # (B, 784)
        y_batch = y_train_oh[i:i+batch_size]    # (B, 10)

        # Forward
        y_pred = predict(X_batch, W, b)         # (B, 10)

        # Loss
        loss = cross_entropy(y_batch, y_pred)

        # Backward
        # TODO: tinh grad_W, grad_b
        pass

        # Update tham so
        # TODO: cap nhat W, b
        pass

    # Accuracy test moi epoch
    y_pred_test = predict(x_test, W, b)         #
    (10000, 10)
    acc_test = np.mean(np.argmax(y_pred_test, axis=1) == y_test)
    print(f"Epoch {epoch+1}/{epochs}, Loss: {loss:.4f}, Test acc:
          {acc_test:.4f}")

# =====
# 7. Evaluation
# =====
def confusion_matrix(y_true, y_pred, num_classes):
    """
    y_true: (N,)
    y_pred: (N,)
    return: (num_classes, num_classes)
    """
    # TODO: tu implement confusion matrix
    pass

```

```
def precision_recall_f1(cm):
    """
    cm: confusion matrix (C, C)
    return: precision, recall, f1 (theo tung lop), f1_macro
    """
    # TODO: tu implement cong thuc
    pass

# Sau khi train xong:
y_pred_labels = np.argmax(predict(x_test, W, b), axis=1) #
                (10000,)
cm = confusion_matrix(y_test, y_pred_labels, num_classes)

prec, rec, f1, f1_macro = precision_recall_f1(cm)

print("Confusion Matrix:\n", cm)
print("Precision per class:", prec)
print("Recall per class:", rec)
print("F1 per class:", f1)
print("Macro F1:", f1_macro)
```

5 Gợi ý công thức (cho sinh viên tự làm)

- Softmax:

$$\text{softmax}(z_i) = \frac{e^{z_i}}{\sum_j e^{z_j}} \quad (1)$$

- Loss (Cross-Entropy):

$$L = -\frac{1}{N} \sum_{i=1}^N \sum_{k=1}^C y_{ik} \log(\hat{y}_{ik}) \quad (2)$$

- Gradient (đạo hàm riêng):

$$\text{grad}_W = (1/N) * X^T(y_pred - y_true) \quad (3)$$

$$\text{grad}_b = (1/N) * \text{sum}(y_pred - y_true) \quad (4)$$

- Cập nhật tham số sử dụng Gradient Descent:

$$W := W - \eta \cdot \text{grad}_W, \quad b := b - \eta \cdot \text{grad}_b \quad (5)$$

Trong đó:

- η (learning rate) là tốc độ học.
- grad_W và grad_b là gradient vừa tính.
- Confusion Matrix ($C \times C$):

- Hàng: nhãn thật
- Cột: nhãn dự đoán
- $cm[i, j]$ = số mẫu thực tế lớp i nhưng dự đoán thành lớp j

• **Precision, Recall, F1 (cho từng lớp k):**

$$\text{Precision}_k = \frac{TP_k}{TP_k + FP_k}, \quad \text{Recall}_k = \frac{TP_k}{TP_k + FN_k}, \quad F1_k = \frac{2 \cdot \text{Precision}_k \cdot \text{Recall}_k}{\text{Precision}_k + \text{Recall}_k} \quad (6)$$

• **Macro F1:**

$$F1_{\text{macro}} = \frac{1}{C} \sum_{k=1}^C F1_k \quad (7)$$